Root Cause Analysis in CARCI

How an anomaly score becomes an explanation someone can act on

Teaching Guide

Six stages, with the code for each one

Cost Anomaly Root Cause Intelligence

Isolation Forest · SHAP · service-day baselines · Google Gemini

Companion to the Complete Code Documentation

Contents

1. What root cause analysis actually means
   - 1.1 The three questions
   - 1.2 Why a score is not a cause
   - 1.3 The six stages
2. Stage 1 - Detect: what is unusual
   - 2.1 The model's contribution
   - 2.2 The two-detector rule
3. Stage 2 - Localise: where and when
   - 3.1 The bug that shaped this stage
   - 3.2 Service-day baselines
   - 3.3 Picking the driver row
4. Stage 3 - Attribute: which measurements drove it
   - 4.1 Reading a SHAP value
   - 4.2 From features to plain language
5. Stage 4 - Decompose: which slice carries the cost
6. Stage 5 - Narrate: facts into sentences
   - 6.1 The template engine
   - 6.2 Severity without a model
7. Stage 6 - Reason: hypotheses about the cause
   - 7.1 The facts/hypotheses split
   - 7.2 What is sent
   - 7.3 What comes back
   - 7.4 Distrusting the reply
   - 7.5 Failing safely
8. A worked example, end to end
9. How to verify any of this yourself
10. What this system cannot do
11. Teaching notes and exercises
    - 11.1 A 60-minute session
    - 11.2 Exercises
    - 11.3 Exam-style questions with answers

1 What root cause analysis actually means

1.1 The three questions

Any anomaly system worth using answers three questions, and they get progressively harder:

Question	Answered by	Difficulty
What is unusual?	A model that scores rows	Solved. Dozens of algorithms do this
Why is it unusual?	Attribution and decomposition	Harder. Needs the model to be interpretable
What caused it, and what do I do?	Reasoning over the evidence	Hardest. The cause is *not in the data*

Most anomaly detectors stop after the first question. That is why so many of them get switched off: an alert that says "row 4,271 is unusual" gives the person reading it no way to decide whether to care.

1.2 Why a score is not a cause

This is the single most important idea in the guide, so it is worth stating bluntly.

> **The cause is never in the billing data**
>
> A billing file records *what you were charged*. It does not record that someone left a test cluster running, or that a deployment introduced a retry loop, or that an instance was compromised. Those facts live in CloudTrail, in a deployment log, in someone's memory — not in the CSV.
>
> So no amount of statistics can extract a root cause from the file. What the statistics can do is narrow the search: identify which service, which day, which resource, which measurements moved. The final step from evidence to cause is an act of *reasoning*, and CARCI treats it as one.

Everything in this guide follows from that. Stages 1 to 4 build evidence, and are exact. Stage 5 puts the evidence into words, and is still exact. Stage 6 reasons from it, and is explicitly uncertain — which is why every hypothesis it produces carries a confidence level and cites the evidence it rests on.

1.3 The six stages

```
a billing CSV
     |
1. DETECT      Isolation Forest scores every row      "row 4,271 is odd"
     |         + a business rule
     v
2. LOCALISE    aggregate to (service, date)           "AmazonEC2, 13 Jan"
     |         compare against that service's
     |         other days
     v
3. ATTRIBUTE   SHAP splits the score across           "usage_quantity,
     |         the 7 features                          cost, transfer"
     v
4. DECOMPOSE   share by region / resource /           "72% in
     |         tag / usage type                         ap-southeast-1"
     v
5. NARRATE     facts become sentences                 a paragraph
     |
     v
6. REASON      ranked hypotheses about the cause      "consistent with
               + how to confirm them                    crypto mining"
```

Sections 2 to 7 take one stage each: what it contributes, the code that does it, and the mistake it exists to prevent. Section 8 follows one real anomaly through all six.

2 Stage 1 — Detect: what is unusual

STAGE 1 of 6 · Input: cleaned billing rows · Output: candidate rows

2.1 The model's contribution

An Isolation Forest scores every line item. The intuition: pick a random feature, pick a random split, cut the data in two, repeat until every point sits alone, then count the cuts each point needed. Points far from everything get isolated in very few cuts. *Anomalies are the ones that are easy to isolate.*

For root cause purposes the important property is not accuracy but *what kind* of anomaly it finds. The forest looks at seven numeric features at once, so it can flag a combination that no single threshold would catch — moderate cost, moderate usage, but an unusual *ratio* between them.

2.2 The two-detector rule

The model alone is not trusted to decide what gets reported. A business rule runs beside it, and the two are joined by OR:

carci/analysis.py - the detection rule

```
147     has_baseline = scored_df["service_day_count"] >= 2
148     model_flagged = scored_df["model_is_anomaly"] == 1
149     large_change = (scored_df["percentage"] >= 75.0) & (scored_df["impact_value"] >= 5.0)
150     candidates = scored_df[has_baseline & (model_flagged | large_change) & (scored_df["impact_value"] >
    0)].copy()
```

Read the last line carefully, because it encodes four separate decisions:

Condition	Why it is there
`has_baseline`	A service with only one day in the file has nothing to be compared against. Reporting it would mean inventing a baseline
`model_flagged OR large_change`	Two independent detectors. The model catches strange combinations a rule could not express; the rule catches large obvious rises the model might miss
`impact_value > 0`	A service-day cheaper than usual is not a cost anomaly worth reporting
`>= 75%` and `>= $5`	The percentage alone would report $0.02 rising to $0.05 — a 150% increase nobody cares about. The dollar floor filters noise the percentage cannot

Teaching point — why OR and not AND

Students almost always suggest AND, reasoning that requiring both detectors would reduce false positives. It would. It would also mean the system reports nothing unless an unsupervised model and a hand-written threshold happen to agree, which throws away the distinct thing each one is good at.

Ask them: which anomalies does each detector catch that the other misses? That question is the whole design.

3 Stage 2 — Localise: where and when

STAGE 2 of 6 · Input: candidate rows · Output: one (service, date) per incident

3.1 The bug that shaped this stage

This stage exists because of a real defect, and the docstring still records it:

carci/analysis.py - _attach_service_context

```
102 def _attach_service_context(scored_df: pd.DataFrame) -> pd.DataFrame:
103     """Attach a date-aware baseline to every scored row.
104 
105     The baseline answers "what does this service normally cost on a day?" by
106     aggregating to the (service, date) level and averaging the service's *other*
107     days. The previous row-level leave-one-out mean ignored dates entirely, so a
108     service that was uniformly expensive looked anomalous on its priciest row
109     even with no change over time.
```

The original code compared each row against the mean of every other row in the file. That sounds reasonable and is badly wrong. Consider a file containing cheap S3 requests and expensive EC2 instances. Every EC2 row sits far above the all-rows mean, so EC2 looks anomalous *constantly* — not because anything changed, but because EC2 is expensive.

> **The question you compare against decides what you find**
>
> **Wrong question:** "is this row expensive compared to all rows?" Answer: yes, it is EC2.
>
> **Right question:** "is AmazonEC2 today more expensive than AmazonEC2 usually is?" Answer: that is an anomaly.
>
> Anomaly detection is comparison, and choosing the comparison group is a modelling decision, not an implementation detail. Most bad anomaly systems fail here.

3.2 Service-day baselines

The fix aggregates to `(service, date)` and builds a leave-one-out mean over that service's *other* days:

carci/analysis.py - building the baseline

```
    scored_df["service_date_cost"] = scored_df.groupby(["service", "date"])["cost"].transform("sum")

    per_day = scored_df.drop_duplicates(subset=["service", "date"])
    day_totals = per_day.groupby("service")["service_date_cost"].agg(["sum", "count"])
    scored_df["service_total_cost"] = scored_df["service"].map(day_totals["sum"])
    scored_df["service_day_count"] = scored_df["service"].map(day_totals["count"])

    scored_df["normal_value"] = np.where(
        scored_df["service_day_count"] > 1,
        (scored_df["service_total_cost"] - scored_df["service_date_cost"])
        / (scored_df["service_day_count"] - 1),
        0.0,
    )
    scored_df["impact_value"] = scored_df["service_date_cost"] - scored_df["normal_value"]
```

Three things to point out in that fragment:

1. `transform("sum")` totals the whole service-day but keeps one value per *row*, so the total can sit alongside the row that drove it.
2. `drop_duplicates` before the per-service totals stops a service-day with 40 rows counting 40 times in the average.
3. The leave-one-out arithmetic — subtract today, divide by the remaining day count — keeps the day being judged out of its own baseline. Leaving it in would drag the baseline toward the anomaly and shrink the very increase you are trying to measure.

3.3 Picking the driver row

A service-day may contain hundreds of line items. Reporting all of them would produce hundreds of identical cards, so one row is chosen to represent the incident:

carci/analysis.py - deduplication and capping

```
    # cards carrying identical breakdowns.
    candidates = candidates.sort_values(
        ["anomaly_score", "cost"], ascending=[False, False]
    ).drop_duplicates(subset=["service", "date"], keep="first")

    total_found = int(len(candidates))
    candidates = candidates.sort_values(
        ["priority_score", "impact_value", "cost"],
        ascending=[False, False, False],
    ).head(MAX_ANOMALIES)
```

The highest-scoring row becomes the *driver*: the specific line item that best explains the day. This is why the UI shows both a "Service Day Cost" and a "Driver Row Cost" — the first is the size of the incident, the second is where to start looking.

4 Stage 3 — Attribute: which measurements drove it

STAGE 3 of 6 · Input: the driver row · Output: per-feature contributions

The forest gives one number per row. SHAP splits that number across the seven features, so instead of "this row scored -0.0512" you get "the score was driven mostly by usage_quantity, then cost, then data_transfer_gb".

4.1 Reading a SHAP value

carci/analysis.py - _get_feature_contributions

```
176 def _get_feature_contributions(row: pd.Series) -> Tuple[List[Dict], List[str]]:
177     feature_rows = []
178     for column in FEATURE_COLUMNS:
179         shap_value = float(row.get(f"shap_{column}", 0.0))
180         feature_rows.append(
181             {
182                 "feature": column,
183                 "feature_value": round(float(row.get(column, 0.0)), 4),
184                 "shap_value": round(shap_value, 4),
185                 "impact_strength": round(abs(shap_value), 4),
186             }
187         )
188
189     feature_rows.sort(key=lambda item: item["impact_strength"], reverse=True)
190     top_features = [item["feature"] for item in feature_rows[:3] if item["impact_strength"] > 0]
191     return feature_rows, top_features
```

Field	Meaning
`feature_value`	What this row actually measured, in its own units
`shap_value`	How far this feature pushed the anomaly score, and in which direction
`impact_strength`	`abs(shap_value)` — how much the feature mattered, ignoring direction

> **Why the ranking uses the absolute value**
>
> Sorting by `impact_strength` rather than by `shap_value` is deliberate. A feature that pushes the score strongly toward "normal" is just as much part of the explanation as one pushing toward "anomalous" — it tells you what was *not* wrong. Ranking by the signed value would bury those.
>
> Note also the `> 0` filter on the last line: a feature with exactly zero contribution explains nothing and is left out of the top list entirely.

> **A caveat to state plainly to students**
>
> SHAP values are attributions of the *model's score*, not of the money. A feature can dominate the SHAP ranking without being where the dollars went. Stage 4 exists precisely because these two questions are different, and confusing them is the most common misreading of an explainability output.

4.2 From features to plain language

`top_ml_features` carries raw column names like `cost_per_usage`. Those are meaningless to a finance reader, so the naming is deferred: the template quotes them as-is, and the language model in stage 6 is instructed to render them in plain language instead. "Unit price" is what `cost_per_usage` means; the pipeline never has to hard-code that translation.

5 Stage 4 — Decompose: which slice carries the cost

STAGE 4 of 6 · Input: the service-day · Output: share by region, resource, tag, usage type

Stage 3 said which *measurements* moved. Stage 4 says which *slice of the business* the money went to, and it is pure arithmetic with no model involved:

carci/analysis.py - _to_breakdown

```python
def _to_breakdown(df: pd.DataFrame, column: str, limit: int = 8) -> List[Dict]:
    if df.empty or column not in df.columns:
        return []

    total_cost = float(df["cost"].sum())
    grouped = (
        df.groupby(column, dropna=False)["cost"]
        .sum()
        .sort_values(ascending=False)
        .head(limit)
        .reset_index()
        .rename(columns={column: "name"})
    )
    grouped["share_pct"] = np.where(total_cost > 0, (grouped["cost"] / total_cost) * 100.0, 0.0)
    grouped["cost"] = grouped["cost"].round(2)
    grouped["share_pct"] = grouped["share_pct"].round(2)
    return grouped.to_dict(orient="records")

```

The same function runs for each of five dimensions, always within the anomalous service-day:

Dimension	The question it answers
`region`	Is this concentrated in one region, or spread everywhere?
`resource`	One runaway instance, or the whole fleet drifting up?
`tag`	Which team or project owns this spend?
`usage_type`	Compute, storage, data transfer, or requests?
`service`	How does this service compare to others on the same day?

> **Concentration is the most useful signal here**
>
> `region_share_pct` at 72% and at 12% mean completely different things. High concentration points at a single misbehaving thing — one instance, one job, one bucket. Low concentration points at something systemic — a price change, a traffic increase, a fleet-wide deployment.
>
> This is why the share percentage, not just the top name, is passed forward to the reasoning stage. "ap-southeast-1" alone would not distinguish those two cases.

6 Stage 5 — Narrate: facts into sentences

STAGE 5 of 6 · Input: all the evidence · Output: a paragraph, a headline, a severity

6.1 The template engine

Before any language model is involved, the evidence is assembled into English by string formatting. The most instructive part is how it words the *flag*:

carci/analysis.py - the three-way flag branch

```
243     if record["is_fallback"]:
244         flag_text = (
245             "No row crossed the anomaly threshold, so this service-day is shown only because it "
246             "carries the largest positive cost impact. Treat it as informational, not as a detection."
247         )
248     elif record["model_flag"]:
249         flag_text = "The Isolation Forest model flagged the driving line item as anomalous."
250     else:
251         flag_text = (
252             "The model did not flag this service-day, but the cost increase exceeded the "
253             "review thresholds (75% above baseline and at least $5 of impact)."
254         )
```

Three branches, because there are genuinely three situations, and collapsing them would be a lie:

Situation	What the text must not claim
`is_fallback` — nothing crossed the threshold	That anything was detected at all. It says "informational, not a detection"
`model_flag` — the forest flagged it	Nothing; this is the honest case
Neither — only the business rule fired	That the model found it. It says so explicitly

> **Teaching point — never overstate the evidence**
>
> It would be easy, and much shorter, to write one sentence saying "the system detected an anomaly" for all three cases. It would also be false in two of them.
>
> A system that tells a user something was machine-detected when it was actually caught by a hand-written `>= 75%` rule has misrepresented its own confidence. Users calibrate their trust on exactly this kind of wording.

6.2 Severity without a model

Severity is computed arithmetically, so it exists whether or not the language model runs:

carci/analysis.py - _severity_for

```
221 def _severity_for(record: Dict) -> str:
222     """Rank an anomaly by money at stake, then by how sharp the rise was.
223 
224     Computed from the pipeline's own figures so that a severity exists even with
225     the LLM switched off. When the model runs it may override this with its own
226     reading, which also weighs the usage type and the feature pattern.
227     """
228     if record["is_fallback"]:
229         # Nothing crossed the threshold; this row is context, not an incident.
230         return "low"
231     impact = record["impact_value"]
232     percentage = record["percentage"]
233     if impact >= 1000 or (impact >= 250 and percentage >= 500):
234         return "critical"
235     if impact >= 250 or (impact >= 50 and percentage >= 200):
236         return "high"
237     if impact >= 50:
238         return "medium"
239     return "low"
240 
```

Note that it takes impact and percentage *together*. A 900% rise on $3 is not an incident; a 40% rise on $8,000 is. Either signal alone would rank the list badly.

The fallback row is forced to `low` regardless of its numbers, for the same reason the template refuses to call it a detection: it is context, not an incident.

7 Stage 6 — Reason: hypotheses about the cause

STAGE 6 of 6 · Input: the evidence as JSON · Output: ranked causes and checks

7.1 The facts/hypotheses split

A root cause report has to do two things that pull in opposite directions. It must quote figures with absolute fidelity, and it must speculate about causes — because, as section 1.2 established, the cause is not in the data.

Treating those as one task is what makes language models dangerous in financial reporting. So the prompt separates them explicitly:

```
EVIDENCE (stages 1-4)          computed by the pipeline
     |                         exact, reproducible, auditable
     |
     +---> FACTS               quoted verbatim, never recomputed
     |                         "2979.72 USD against a baseline of 446.67"
     |
     +---> HYPOTHESES          reasoned, ranked, uncertain
                               "consistent with a compromised instance"
                               confidence: high
                               evidence:   usage_type is CryptoMiningTraffic
```

carci/llm_reasoning.py - the HYPOTHESES rules

```
78  HYPOTHESES - what plausibly caused the anomaly.
79
80    * Here you are expected to reason past the data and propose specific engineering
81      explanations. This is the part of the report a human analyst would write.
82    * Ground every hypothesis in something you were given: the usage type, the
83      resource, how concentrated the region share is, the tag, and which features
84      the attribution step found most important.
85    * Rank them, and say how confident you are. A hypothesis you cannot tie to
86      anything in the input is not worth listing.
87    * Never state a hypothesis as established fact. "Consistent with", "suggests"
88      and "would explain" are correct. "Was caused by" is not.
```

	FACTS	HYPOTHESES
Source	The pipeline	The model's reasoning
May the model invent?	Never	That is the point
Certainty	Exact	Ranked, with a confidence
Must cite evidence?	It *is* the evidence	Yes, a named field
Wording	Plain assertion	"consistent with", never "was caused by"

> **Why this is the right division of labour**
>
> The model is bad at arithmetic and good at pattern-matching against a large body of engineering knowledge. So it is given zero arithmetic and asked for exactly the thing it is good at: *given these symptoms, what conditions typically produce them?*
>
> That is also what a human analyst does. They do not recompute the bill; they read the evidence and propose what would explain it.

7.2 What is sent

Only a narrow, named set of fields leaves the pipeline:

carci/llm_reasoning.py - _to_payload

```
219 def _to_payload(record: Dict, index: int) -> Dict:
220     """Reduce an anomaly record to the facts the model is allowed to talk about.
221 
222     Deliberately narrow: passing the whole record would hand the model fields it
223     has no business repeating, and every extra key is a chance for it to state
224     something the pipeline never computed. No suggested action is included, so
225     the recommendation the model returns is its own reading of these facts rather
226     than a rewrite of a keyword rule.
227     """
```

The narrowing is a correctness decision, not a bandwidth one: every extra key is a chance for the model to repeat something the pipeline never computed. Note also what is *absent* — the keyword-based recommendation is withheld, so the action the model returns is its own reading of the evidence rather than a paraphrase of a rule.

7.3 What comes back

The reply is constrained by an explicit `responseSchema`, so it arrives as parseable JSON rather than prose to be scraped:

Field	Kind	Purpose
`headline`	Hypothesis	Names the suspected cause, not the symptom
`severity`	Judgement	critical / high / medium / low
`explanation`	Facts	Three or four paragraphs of narrative
`likely_causes`	Hypothesis	Ranked, each with `confidence` and `evidence`
`investigation_steps`	Method	Checks that confirm or eliminate a cause
`recommended_action`	Judgement	What to do once confirmed

The investigation steps are what make it falsifiable

A hypothesis you cannot check is just an opinion. Requiring each anomaly to come with the specific console page, log or command that would settle the question turns the output from a guess into something a reader can act on and disprove.

This is the difference between "probably crypto mining" and "open CloudTrail for this instance in ap-southeast-1 and look for unexpected inbound connections".

7.4 Distrusting the reply

The schema guarantees the *shape* of the reply. It guarantees nothing about the content, so the first code that touches the response validates it:

carci/llm_reasoning.py - _clean_causes

```
335 def _clean_causes(value: object) -> List[Dict[str, str]]:
336     """Keep only hypotheses that carry both a mechanism and its evidence.
337 
338     A cause with no evidence is exactly the unsupported speculation the prompt
339     forbids, so it is discarded here rather than shown to the user.
340     """
341     if not isinstance(value, list):
```

A hypothesis missing either its mechanism or its evidence is discarded outright. That is the rule from 7.1 enforced in code rather than merely requested in a prompt — unsupported speculation never reaches the screen.

Teaching point — validate at the boundary

The schema is enforced by someone else's server. These cleaners are the first code *you* control that touches the reply, and they are where "the response is well-formed" gets checked instead of assumed.

Treat every external API this way, including ones whose contract you believe you have pinned down. A prompt is a request, not a guarantee.

7.5 Failing safely

Five different things can stop this stage producing output: no API key, the feature switched off, a network failure, an HTTP error, or an unparseable reply. All five produce the same outcome — the stage-5 template text is kept and the app carries on:

carci/analysis.py - _apply_llm_explanations, the safe part

```
    for record in records:
        record["template_explanation"] = record["root_cause_analysis"]
        record["template_recommendation"] = record["recommendation"]
        record["template_headline"] = record["headline"]
        record["explanation_method"] = "deterministic template"
        # Hypotheses and investigation steps have no template equivalent: naming a
        # probable cause is a judgement no keyword rule can make. In template mode
        # these stay empty and the UI simply omits those sections.
        record["likely_causes"] = []
        record["investigation_steps"] = []

    generated = generate_explanations(records)
    if not generated:
```

The template is copied into `template_*` fields *before* any network call, so it cannot be lost. `likely_causes` and `investigation_steps` start empty because they have no template equivalent: naming a probable cause is exactly the judgement no keyword rule can make. Offline, those sections are simply absent rather than filled with something weaker.

> **Graceful degradation has a cost you must design for**
>
> A silent fallback is invisible. A student demonstrating this project with an expired key would see plausible explanations and never notice the model had not run.
>
> That is why `explanation_method` is recorded on every record and printed in the UI. **Degrade quietly, but always leave a way to find out that you did.**

8 A worked example, end to end

One anomaly, followed through all six stages. Every number below is real output from `dataset/test_2_obvious_anomalies.csv`.

The evidence the pipeline built (stages 1–5)

```
STAGE 1  DETECT
   model_is_anomaly = 1          the forest flagged the driver row
   percentage       = 567.1      the business rule would have caught it too

STAGE 2  LOCALISE
   service_date_cost = 2979.72   AmazonEC2, 13 January 2026
   normal_value      =  446.67   mean of EC2's other days in the file
   impact_value      = 2533.05   2979.72 - 446.67
   driver_row_cost   = 1011.65   the single line item that scored highest

STAGE 3  ATTRIBUTE
   usage_quantity     value 16049.04    shap -2.3399
   cost               value  1011.65    shap -2.2855
   data_transfer_gb   value   770.18    shap -2.2010

STAGE 4  DECOMPOSE
   region      ap-southeast-1    72.01% of the service-day
   resource    db-1cd8dd14-...
   usage_type  CryptoMiningTraffic
   tag         AIResearch

STAGE 5  NARRATE
   severity    critical          impact >= 1000
   headline    "AmazonEC2 CryptoMiningTraffic up 567% in ap-southeast-1"
```

What stage 6 made of it

```
headline   Crypto-mining traffic on EC2 instance db-1cd8dd14
severity   critical

explanation
   On 13 January 2026, AmazonEC2 spend surged by 567.10% to reach an observed
   cost of 2979.72 USD, generating an impact of 2533.05 USD over a baseline of
   446.67 USD. A single line item contributed 1011.65 USD.

   The cost was heavily concentrated in the ap-southeast-1 region, which
   accounted for 72.01% of the regional share, tied to resource db-1cd8dd14-...
   under the AIResearch tag. The usage type was explicitly flagged as
   CryptoMiningTraffic, indicating unauthorized or unintended workload execution.

   Feature attribution highlights usage quantity at 16049.04, total cost at
   1011.65 USD, and data transfer of 770.18 GB as the primary drivers behind the
   anomaly score. These metrics reflect high throughput and prolonged compute
   utilization consistent with cryptocurrency mining operations.

   (fourth paragraph: escalation)

likely_causes
   1. [high]   Compromised EC2 instance running unauthorized cryptocurrency
               mining scripts
      evidence usage_type is CryptoMiningTraffic with 16049.04 usage quantity
               and 770.18 GB data transfer

   2. [medium] Unintended deployment of heavy proof-of-work algorithms under the
               AIResearch tag
      evidence Tag is AIResearch and ap-southeast-1 holds 72.01% region share

investigation_steps
   1. Open the Amazon EC2 Console and inspect resource db-1cd8dd14-... in
      region ap-southeast-1.
   2. Review VPC Flow Logs and CloudTrail logs for unexpected inbound
      connections or credential misuse on the instance.
   3. Check security group rules associated with the resource to identify
      overly permissive access.
```

Now check the model's work

This is the exercise worth doing with students. Take every number in the generated prose and find it in the evidence block above:

Number in the prose	Where it came from	Invented?
567.10%	`percentage`, stage 2	No
2979.72 USD	`service_date_cost`, stage 2	No
2533.05 USD	`impact_value`, stage 2	No
446.67 USD	`normal_value`, stage 2	No
1011.65 USD	`driver_row_cost`, stage 2	No
72.01%	`region_share_pct`, stage 4	No
16049.04, 770.18	SHAP feature values, stage 3	No

Every figure traces back. Automated over the full 16-anomaly run, 453 numbers were checked and none was unmatched.

> **Now do the same to the hypotheses — and notice the difference**
>
> "Compromised EC2 instance running unauthorized mining scripts" appears nowhere in the evidence. It *cannot*, because compromise is not a billing fact.
>
> That is not a failure — it is stage 6 doing its job. The controls that make it responsible are that it is labelled a hypothesis, ranked second-to-none rather than asserted, given a confidence of `high` rather than certainty, tied to the specific fields that motivated it, and shipped with three checks that would prove it wrong.
>
> **The lesson: separate what you know from what you infer, and label which is which.** That principle is older than machine learning and applies well beyond this project.

9 How to verify any of this yourself

Because the fallback is silent, "the explanations look good" is not evidence that the reasoning stage ran. Three checks, in increasing order of certainty:

1. Look at the caption in the app

Page 3, under the "Root Cause Analysis" heading. It reads either "*Explanations written by LLM (gemini-3.5-flash-lite)*" or "*Explanations from the deterministic template*".

2. Look at the card

Ranked causes and investigation steps appear **only** in LLM mode. If a card shows a narrative and a severity but no "Likely causes" section, stage 6 did not run.

3. Inspect the result object

```
import pandas as pd
from carci.analysis import build_analysis_result
from carci import llm_reasoning

result = build_analysis_result(pd.read_csv("dataset.csv", skipinitialspace=True))

# Which engine wrote the text?
print(result["metadata"]["explanation_method"])
# -> "LLM (gemini-3.5-flash-lite)"   or   "deterministic template"

# If it fell back, why?
print(llm_reasoning.last_error())

record = result["service_comparison"][0]
print(record["severity"], "|", record["headline"])
for cause in record["likely_causes"]:
    print(cause["confidence"], "-", cause["cause"])
    print("   evidence:", cause["evidence"])

# The template is always kept, so the two can be compared side by side
print(record["template_explanation"])
```

Forcing each state for a demonstration

To show	Do this
The full six-stage output	A valid `GEMINI_API_KEY` in `.env`
The template fallback	Set `CARCI_LLM_REASONING=0` and re-run
A diagnosable failure	Corrupt the key. `last_error()` reports the cause; the app still works

A demonstration worth doing in front of a class

Run the same file twice, once with the key and once with `CARCI_LLM_REASONING=0`, and put the two cards side by side.

The numbers are identical. The severity is identical. Only the wording and the hypotheses differ. That single comparison teaches the architecture better than any diagram: it shows exactly which parts of the output the language model is, and is not, responsible for.

10 What this system cannot do

State these plainly. A report that lists its own limits is more credible, not less.

1. **It cannot confirm a cause.** Stage 6 produces hypotheses. Confirmation requires the investigation steps to actually be run against CloudTrail, the console or the logs — none of which CARCI reads.
2. **It misses about a third of anomalies.** Recall is 0.657 on the labelled dataset. The explanations are only as good as the detection that precedes them.
3. **It always flags roughly 5% of rows.** `contamination = 0.05` sets where the cut-off sits, not how many anomalies exist. A perfectly clean file still produces flags.
4. **SHAP attributes the score, not the money.** The most important feature is not necessarily where the dollars went.
5. **Prompt adherence is not enforced.** The schema guarantees the reply's structure; nothing validates that the prose obeys the writing rules.
6. **The reasoning stage is not reproducible.** The same file analysed twice can produce different wording and different hypotheses. The facts do not change; the narrative does.
7. **Billing details leave the machine.** Resource identifiers, tags and figures are sent to a third-party API. `CARCI_LLM_REASONING=0` exists so this can be turned off.

> **The honest summary**
>
> CARCI narrows a 10,000-row bill down to twenty incidents, tells you exactly what changed and where, and proposes what to check first. It does not tell you what happened. A human still closes that gap — the system's job is to make the gap small enough to close quickly.

11 Teaching notes and exercises

11.1 A 60-minute session

Time	Topic	Material
0–10 min	Why a score is not a cause	Section 1.2. Ask what a billing file does *not* record before showing the answer
10–20 min	Detection and localisation	Sections 2 and 3. The all-rows baseline bug is the strongest teaching moment in the project
20–30 min	Attribution and decomposition	Sections 4 and 5, emphasising that they answer different questions
30–45 min	The facts/hypotheses split	Section 7.1, then the worked example in section 8
45–55 min	Live demo	Run with and without the key, cards side by side
55–60 min	Limits	Section 10. Close on what the system cannot do

11.2 Exercises

1. Change the business rule from 75% to 40% and re-run. How many more anomalies appear, and how many are worth acting on? Argue for a threshold from the result.
2. Remove the `impact_value >= 5.0` floor. Find an anomaly that now appears which nobody would care about, and explain in one sentence why the percentage alone was misleading.
3. Set `CARCI_LLM_REASONING=0` and write down three things the report loses. For each, say whether it could be recovered by a rule.
4. Take one generated explanation and verify every number against `record["feature_contributions"]` and the record fields. Report anything that does not trace.
5. Write a hypothesis for an anomaly yourself, before reading the model's. Compare. Which of you cited better evidence?
6. The system caps output at 20 anomalies. Design an alternative ranking to `priority_score` and justify it in terms of what a finance team actually needs first.
7. Propose a way to measure whether the hypotheses are any good. This is genuinely hard: there are no labels for causes. A serious answer is worth more than a confident one.

11.3 Exam-style questions with answers

Q1. Why can a root cause not be computed from the billing file alone?

The file records what you were charged, not why. Compromise, misconfiguration and deployment mistakes leave no column in a CUR export. Statistics can localise the anomaly to a service, day, region and resource, but the final step to a

cause is inference from evidence, not a calculation over it.

Q2. The detection rule joins two detectors with OR. Why not AND?

They catch different things. The forest catches unusual combinations across seven features that no threshold expresses; the rule catches large obvious rises the forest may score as unremarkable. AND would report only cases where both agree, discarding what each is individually good at.

Q3. Why compare a service-day against that service's other days rather than against all rows?

Because comparing against all rows measures how expensive a service is, not whether it changed. An always-expensive service would be flagged permanently. The comparison group defines what counts as an anomaly, so choosing it is a modelling decision.

Q4. Why does the baseline exclude the day being judged?

Including it would drag the baseline toward the anomaly, shrinking the measured increase. The leave-one-out mean keeps the comparison independent of the value being tested.

Q5. Why is the feature ranking sorted by the absolute SHAP value?

Because a feature pushing strongly toward normal is part of the explanation too. Sorting by the signed value would rank those last and hide what was not wrong.

Q6. A card says severity critical but the model never flagged the row. Is that a contradiction?

No. Severity is computed from impact and percentage, so the business rule alone can produce a critical entry. The narrative states explicitly that the model did not flag it, which is why the three-way branch in the template exists.

Q7. What is the difference between the `explanation` field and the `likely_causes` field?

`explanation` restates facts the pipeline computed and may contain no invented figure. `likely_causes` is inference, is ranked, carries a confidence, and must cite the evidence it rests on. They are held to opposite standards on purpose.

Q8. Why does the code discard a hypothesis that has no evidence field?

Because a cause with nothing behind it is the unsupported speculation the prompt forbids. The prompt is a request; the cleaner is the enforcement. Never rely on an external service to obey your instructions.

Q9. Both engines produce a headline. What can the template's headline never do?

Name a probable cause. It can only describe the symptom, because identifying a cause is a judgement no keyword rule can make. This is the clearest illustration of what the reasoning stage actually adds.

Q10. A user reads a root cause paragraph and finds it useful. What evidence would you need before concluding the language model produced it?

The caption naming the method, the presence of the likely-causes section, or `result["metadata"]["explanation_method"]`. Usefulness is not evidence: the template is designed to be useful too, which is exactly why the fallback is safe and exactly why it is invisible.